COMM2020 Team Project
Project Specification (Project 8)
Food Waste Rescue Marketplace and Demand Forecaster

Usage of AI Tools: AI-Minimal

These tasks have been set to assess your problem-solving abilities, and so you may not use AI for generating code or autocomplete while completing the project.

The University of Exeter is committed to the ethical and responsible use of Generative AI (GenAI) tools in teaching and learning, in line with our academic integrity policies where the direct copying of AI-generated content is included under plagiarism, misrepresentation and contract cheating under definitions and offences in TQA Manual Chapter 12.3.

This assessment falls under the category of AI-Minimal in the University's Guidance on use of Gen AI in Assessment.

This means: You may use AI tools for checking spelling and grammar mistakes only, with no other impact on the structure or content of the assessment. This is because using GenAI tools outside of these uses prevents fair assessment of your ability to achieve module learning outcomes.

When writing your assessment, you must never use AI tools:

- For uses other than checking your spelling and grammar.
- To translate more than a word or short phrase into English.
- To upload sensitive or identifying material to an AI tool.
- To present material that has been generated by AI as your own work or the work of someone else.

When submitting your assessment, you must:

- Check the box during the submission process that confirms you have adhered to the university's academic conduct policy and the expectations on use of GenAI in your assessment brief.

NOTICE: Your coding activity must all be completed on the GitHub repository. This logs your activity as you code, and the logs will be checked for submissions that have made use of AI-generated code.

Guidelines

- This assessment is AI-Minimal: spelling/grammar checks only; no AI code generation or autocomplete.
- Your application will be demonstrated online. Provide a deployed URL for the live demo and a clear local run guide for markers.
- All forecasting must be transparent and explainable (no 'black box' claims). Include baseline models and clearly explain assumptions.
- Your system must remain usable with a seeded dataset even if external services are unavailable.
- If you use third-party services (payments, maps), handle failures gracefully and avoid embedding secrets (use environment variables).

1 Requirements

Build a web application that enables sellers to post surplus food bundles and consumers to reserve/claim them. The system must also provide a seller-facing analytics view, including demand forecasting to reduce future waste.

1.1 The practical problem

Food waste is costly and environmentally damaging. Sellers often over-produce because they lack demand visibility and are wary of stock-outs. Consumers may be willing to buy discounted surplus items, but they need timely, trustworthy availability information.

Your system addresses this by providing:

- A marketplace for surplus bundles with clear pickup windows.
- Inventory and reservation management to prevent overselling.
- Seller analytics showing waste, sell-through rates, and recommended production adjustments.
- Forecasting to estimate demand for bundles by day/time/product type.

1.2 The game concept: Rescue Streaks (required)

To motivate engagement, your system must include a light gamification layer that rewards consumers for consistent waste-rescue behaviour.

- Rescue streaks: consecutive weeks with at least one rescued bundle.

- Badges for variety (rescuing from different sellers or categories) and impact (CO2e saved estimates).
- A personal impact summary (e.g., estimated meals rescued, CO2e saved).

Leaderboards are optional; personal progress tracking is mandatory.

2 Users and user profiles

Your application must support three user types:

2.1 Consumers

- Browse bundles by time and location; reserve a bundle; receive a claim code.
- Track streaks and personal impact.
- Report issues (e.g., bundle unavailable, quality concerns).

2.2 Sellers (trusted users)

- Create and manage bundles (quantity, pickup time, price, contents description, allergens).
- Manage reservations and mark pickups as completed/no-show.
- View analytics and forecasting outputs; record actions taken (e.g., reduced production).

2.3 Maintainers (developers)

- Maintain deployments, data pipelines, and testing.

3 Features

3.1 Marketplace workflow (required)

1. Bundle posting: sellers create bundles with pickup windows and quantity limits.
2. Reservation and claim codes: consumers reserve; system generates a claim code and prevents overselling.
3. Pickup completion: seller verifies claim code and marks as collected; record no-shows after window ends.
4. Consumer notifications: basic on-screen reminders for pickup windows (email optional).
5. Issue reporting: consumers submit issue reports; sellers respond and mark resolved.

3.2 Seller analytics (required)

- Sell-through: proportion of posted bundles collected vs no-show vs expired.
- Waste proxy: estimate waste avoided based on collected bundles and assumed average weight per bundle (document assumptions).
- Pricing effectiveness: compare sell-through at different discount levels.
- Operational insights: identify best pickup windows and most popular categories.

3.3 Forecasting component (required)

You must implement forecasting that estimates demand for bundles. Forecasts must be explainable and compared against baselines.

- Inputs: day of week, time window, seller, category, weather flag (synthetic allowed), and price/discount level.
- Outputs: predicted reservations and expected no-show probability for each posting.
- Baselines: include at least two baseline approaches (e.g., moving average and seasonal naive) and compare against your chosen approach using error metrics.
- Recommendation: provide seller recommendations (e.g., 'post 12 bundles instead of 18 for Tuesday 17:00–18:00'), with rationale and confidence.

3.4 Technical requirements (required)

- Authentication: role-based access control (consumer/seller).
- Security: secrets via environment variables; no secrets in repository or ELE ZIP; do not collect payment card details.
- Accessibility: keyboard navigation for core screens, readable contrast, and mobile-friendly layouts.
- Professional practice evidence: use GitHub commits/issues/PRs; the repository URL must be included in 0_admin/submission.txt.
- Testing (mandatory): you must submit BOTH (A) an automated test suite that can be executed by the markers, AND (B) a manual end-to-end test plan with results evidence.
- Automated tests (A) must include at least 15 automated tests covering: posting/reserving bundles, claim code verification, no-show/expiry logic, analytics calculations, and forecasting pipeline correctness.
- Manual tests (B) must include at least 8 end-to-end scenarios (happy path + failure cases). Include expected results and screenshots/logs of completed runs in 4_technical/testing_evidence.pdf.
- Your deployment_guide.pdf must include a 'How to run tests' section with the exact commands/steps.

4 Data and seeded dataset

You must include a seeded dataset so the system works during marking without requiring external services. The marketplace and forecasting must work using seeded data.

4.1 Minimum dataset contents (required)

- At least 25 sellers and 250 bundle postings seeded across at least 6 weeks.
- At least 400 reservations seeded, including at least 80 no-shows and 50 expiries.
- At least 150 issue reports seeded with statuses and seller responses.

- At least 6 product categories and at least 10 pickup time windows.
- A historical demand dataset for forecasting (can be derived from the seeded reservations).

4.2 Suggested entities (example)

Entity	Key fields (minimum)
Seller	seller_id, name, location, opening_hours, contact_stub
BundlePosting	posting_id, seller_id, category, contents, allergens, quantity, price, pickup_window, status
Reservation	reservation_id, posting_id, consumer_id, timestamp, claim_code, status(reserved/collected/no-show/expired)
Consumer	consumer_id, display_name (non-identifying), streak, badges
IssueReport	issue_id, posting_id, consumer_id(optional), type, description, status, seller_response
ForecastInput	record_id, features(day/time/seller/category/price/weather_flag), observed_reservations, observed_no_show
ForecastOutput	output_id, posting_id, predicted_reservations, predicted_no_show_prob, confidence, rationale

5 Measures of success (evaluation)

You must define and report success measures with evidence. Suitable measures include:

- Marketplace reliability: oversell rate must be 0; correct handling of expiries and no-shows.
- Forecast accuracy: compare your forecasting approach against baselines using clear error metrics and a held-out evaluation period.
- Waste impact: report estimated waste avoided and explain assumptions/limitations.
- User experience: task-based evaluation for reserving and claiming within a time limit.

6 Process and deliverables

You must submit all group deliverables on ELE as a single ZIP. GitHub is used to evidence professional practice and must be referenced in submission.txt.

Intended Learning Outcomes assessed by the coursework

Coursework 1 and Coursework 2 assess all module Intended Learning Outcomes (ILOs):

- ILO1 – Function effectively as a member of a team.
- ILO2 – Apply an integrated or systems approach to the solution of complex problems.
- ILO3 – Apply knowledge of domain context, project and change management, and relevant legal matters including intellectual property rights.
- ILO4 – Select and apply appropriate materials, technologies, and processes, and recognise their limitations.

- ILO5 – Plan self-learning and development to support the activity of the wider team.
- ILO6 – Support an inclusive approach to teamwork and problem solving, recognising the responsibilities, benefits and importance of supporting equality, diversity, and inclusion.
- ILO7 – Evaluate the environmental and societal impact of solutions to complex problems and minimise adverse impacts.

Evidence is expected across reports, implementation, evaluation, ethical/legal materials, and teamwork/process artefacts in both sprints.

6.1 Sprint 1 (Coursework 1) – Prototype v1, report and demonstration (week 5)

Submission to ELE is a single ZIP file named GroupX_CW1.zip.

Sprint 1 deliverable expectations

- Prototype v0.1.0 (vertical slice): seller posts bundle → consumer reserves → claim code generated → seller marks collected → consumer streak updates.
- Analytics: at least 2 seller analytics views implemented with seeded data.
- Forecasting: implement at least one baseline forecast and show evaluation on seeded history (even if simple in CW1).
- Testing evidence (CW1): include at least 5 automated tests running successfully and document how to run them in deployment_guide.pdf; include one manual end-to-end test run with evidence in testing_evidence.pdf.
- Prototype report must include: executive summary, prioritised requirements, architecture v1, forecasting plan, initial evaluation evidence, and a sprint plan for CW2.
- Ethical/legal considerations must cover food safety disclaimers, allergy information handling, consumer fairness, accessibility, and IP/licensing implications.
- Software/data inventory must list all dependencies and datasets (licence, provenance, cost model, versions).

Live demo and presentation (10 minutes) – what to cover (CW1)

6. 30 seconds: what waste problem you solve and how the marketplace works.
7. 6–7 minutes: demo posting, reserving, claim code verification, and streak update.
8. 1–2 minutes: show one analytics view and the baseline forecasting evaluation snapshot.
9. Final minute: what will be completed in Sprint 2 and your top risks.

6.2 Sprint 2 (Coursework 2) – Final prototype, client handover and presentation (week 11)

Submission to ELE is a single ZIP file named GroupX_CW2.zip.

Individual reflection (submitted separately by each student on ELE): reflection.pdf (800–1,000 words; includes evidence links to commits/issues/PRs; and an AI-Minimal compliance statement).

Sprint 2 deliverable expectations

- Final prototype v1.0.0: robust marketplace with issue reporting; full analytics set; complete streak/badge system.
- Forecasting: implement your chosen forecast approach and compare against both baselines on a held-out period; include recommendation logic with confidence labels.
- Client handover pack: clear deployment, operations, and maintenance guidance so another team could run and extend the system.
- Testing evidence (CW2): meet the full testing requirement (15+ automated tests + 8+ manual scenarios) and include clear pass/fail evidence in testing_evidence.pdf. Marks will be reduced if tests cannot be run by markers.
- Final evaluation: report success measures with method and limitations; include environmental impact discussion and unintended consequences (e.g., inequity of access).
- Updated ethical/legal and licensing materials consistent with the final system and all dependencies.

Live demo and presentation (10 minutes) – what to cover (CW2)

10. 1 minute: recap problem and what is now delivered.
11. 6–7 minutes: demo end-to-end marketplace plus analytics and forecasting recommendations.
12. 1–2 minutes: show handover pack and how a maintainer would add a new seller/category and update forecasting features.
13. Final minute: evaluation highlights, limitations, and next steps.

Individual reflection (Coursework 2 – individual deliverable)

Each student must submit an individual reflection on ELE (not in the group repository). Suggested length: 800–1,000 words. This reflection is used to evidence individual learning and contribution and may be used to resolve contribution disputes.

14. Your role and contributions: describe what you owned (features, testing, documentation, deployment). Reference concrete evidence (PR links, issue IDs, commits).
15. What you learned: at least three specific technical or professional learning points linked to module outcomes (e.g., requirements negotiation, risk management, testing, deployment).
16. Challenges and how you addressed them: one technical challenge and one teamwork/process challenge; what changed as a result.
17. Responsible computing: what ethical/legal risk you personally focused on (privacy, accessibility, safety) and how you mitigated it.

18. AI-Minimal compliance statement: confirm you adhered to the brief and did not use GenAI for code generation or content generation beyond spelling/grammar checks.

7 Safety and responsible use (must comply)

- Do not represent bundles as safe to eat without disclaimers; include allergy information and a clear 'consume at your own risk' notice.
- Do not include real payment processing; reservations only.
- Avoid collecting sensitive personal data; use pseudonymous display names.

8 Marking Rubric

The same COMM2020 marking rubric applies across all project options. The rubric will be provided by the module team and used consistently for CW1 and CW2 (including the individual reflection in CW2).

[END OF SPECIFICATION]

Document owner: Module team (COMM2020)

Date: 23 December 2025